

## 3522 BULUT BILISIM DERSI

# PROJE 1 RAPORU

# TaskFlow

Çift Katmanlı Web Uygulaması (Web API + Frontend)

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| Video Linki | <a href="https://youtu.be/gRsnNLLEQ5k">https://youtu.be/gRsnNLLEQ5k</a>           |
| Öğrenci Adı | Şeyma Kula                                                                        |
| Ders        | 3522 Bulut Bilişim                                                                |
| Platform    | Amazon Web Services (AWS)                                                         |
| GitHub      | <a href="https://github.com/seymakula/taskflow">github.com/seymakula/taskflow</a> |
| Tarih       | 1 Nisan 2026                                                                      |

## 1. PROJE OZETI

TaskFlow, Amazon Web Services (AWS) bulut platformu üzerinde geliştirilmiş, çift katmanlı mimariye sahip modern bir görev yönetim web uygulamasıdır. Bu proje, 3522 Bulut Bilişim dersi kapsamında, RESTful API backend ve React frontend entegrasyonunun bulut ortamında nasıl gerçekleştirildiği pratikte göstermek amacıyla geliştirilmiştir.

Uygulama, kullanıcıların görev oluşturabileceği, düzenleyebileceği, tamamlayabileceği ve silebileceği bir görev yönetim panelidir. Bunlara ek olarak, odaklanmayı artırmak amacıyla Pomodoro tekniğine dayalı, kullanıcının süre belirleyebildiği bir zamanlayıcı da uygulamaya entegre edilmiştir. Arayüz dark mode tasarımıyla sunulmakta olup istatistik kartları sayesinde görevlerin özet durumu anlık olarak takip edilebilmektedir.

Projenin temel hedefleri şunlardır:

- Bulut platformlarında uygulama barındırma ve yönetimi
- Yüksek erişilebilirlik ve güvenlik sağlama
- RESTful API ile veri yönetimi
- Modern frontend teknolojileri ile kullanıcı dostu arayüz geliştirme

## 2. KULLANILAN TEKNOLOJILER

### 2.1 Backend Teknolojileri

| Teknoloji        | Versiyon | Kullanım Amacı                |
|------------------|----------|-------------------------------|
| Python           | 3.9.6    | Ana programlama dili          |
| Flask            | 3.1.3    | Web framework, API geliştirme |
| Flask-SQLAlchemy | 3.1.1    | ORM, veritabanı işlemleri     |
| Flask-CORS       | 6.0.2    | Cross-Origin Resource Sharing |
| Gunicorn         | 25.3.0   | Production WSGI sunucusu      |
| psycopg2-binary  | 2.9.11   | PostgreSQL bağlantı sürücüsü  |
| python-dotenv    | 1.2.1    | Environment variable yönetimi |

### 2.2 Frontend Teknolojileri

| Teknoloji         | Versiyon | Kullanım Amacı                  |
|-------------------|----------|---------------------------------|
| React             | 18.x     | Frontend JavaScript kütüphanesi |
| JavaScript (ES6+) | -        | Ana programlama dili            |
| CSS3              | -        | Arayüz tasarımı, dark mode      |
| Fetch API         | -        | HTTP istekleri                  |

## 2.3 AWS Servisleri

| AWS Servisi | Tip             | Kullanım Amacı                |
|-------------|-----------------|-------------------------------|
| Amazon EC2  | t3.micro        | Flask API sunucusu barındırma |
| Amazon RDS  | db.t3.micro     | PostgreSQL veritabanı         |
| Amazon S3   | General Purpose | React frontend static hosting |

### 3. SISTEM MIMARISI

TaskFlow projesi, modern web uygulamalarında yaygın olarak kullanılan uçlu katmanlı mimari (3-tier architecture) üzerine inşa edilmiştir. Her katman birbirinden bağımsız olarak çalışır ve AWS'nin farklı servisleri üzerinde barındırılır.

#### 3.1 Mimari Akış

##### MIMARI AKIS SEMASI

Kullanici --> AWS S3 (React) --> AWS EC2 (Flask API) --> AWS RDS (PostgreSQL)

- Adım 1: Kullanıcı tarayıcısında S3 URL'ini açar
- Adım 2: React, EC2'deki Flask API'ye HTTP isteği gönderir (port 5000)
- Adım 3: Flask isteği isler ve RDS PostgreSQL'e sorgu gönderir (port 5432)
- Adım 4: Veritabanı sonucu JSON olarak React'a döner
- Adım 5: React arayüzü günceller

#### 3.2 Katmanlar Arası İletişim

Frontend ile backend arasındaki iletişim HTTP protokolü üzerinden gerçekleşir. React uygulaması, Flask API'nin public IP adresine fetch API kullanarak istek gönderir. Flask-CORS middleware'i sayesinde farklı origin'lerden gelen isteklere izin verilmiştir.

Backend ile veritabanı arasındaki iletişim ise güvenli bir VPC (Virtual Private Cloud) içinde, PostgreSQL protokolü üzerinden sağlanır. RDS instance'i, yalnızca tanımlanan güvenlik grubu kuralları çerçevesinde dış bağlantılara izin verir.

## 4. AWS SERVİS YAPILANDIRMASI

### 4.1 Amazon EC2 Yapılandırması

EC2 (Elastic Compute Cloud), Flask API'nin çalıştığı sanal sunucudur. Aşağıdaki parametrelerle yapılandırılmıştır:

| Parametre      | Değer                                                |
|----------------|------------------------------------------------------|
| Instance Adi   | taskflow-server                                      |
| AMI            | Ubuntu Server 24.04 LTS                              |
| Instance Tipi  | t3.micro (Free Tier)                                 |
| Bolge          | eu-central-1b (Frankfurt)                            |
| Public IP      | 3.120.139.109                                        |
| Guvencik Grubu | taskflow-sg                                          |
| Acik Portlar   | 22 (SSH), 80 (HTTP), 5000 (Flask), 5432 (PostgreSQL) |

### 4.2 Amazon RDS Yapılandırması

RDS (Relational Database Service), PostgreSQL veritabanının barındırıldığı AWS yönetimli veritabanı servisi

| Parametre      | Değer                                                   |
|----------------|---------------------------------------------------------|
| DB Identifier  | taskflow-db                                             |
| Engine         | PostgreSQL 17                                           |
| Instance Tipi  | db.t3.micro (Free Tier)                                 |
| Depolama       | 20 GB SSD (gp2)                                         |
| Bölge          | eu-central-1b (Frankfurt)                               |
| Endpoint       | taskflow-db.cdewe2imkv9k.eu-central-1.rds.amazonaws.com |
| Port           | 5432                                                    |
| Veritabanı Adı | taskflowdb                                              |
| Kullanıcı Adı  | taskflow_user                                           |

### 4.3 Amazon S3 Yapılandırması

S3 (Simple Storage Service), React uygulamasının production build dosyalarının barındırıldığı servistir:

| Parametre      | Değer                                                             |
|----------------|-------------------------------------------------------------------|
| Bucket Adı     | taskflow-frontend-seymakula                                       |
| Bölge          | eu-central-1 (Frankfurt)                                          |
| Static Hosting | Aktif                                                             |
| Index Document | index.html                                                        |
| Website URL    | taskflow-frontend-seymakula.s3-website.eu-central-1.amazonaws.com |

## 5. BACKEND GELISTIRME (FLASK API)

### 5.1 Proje Yapısı

Backend, MVC benzeri bir yapı ile organize edilmiştir:

```
taskflow/backend/
|-- app.py          # Uygulama baslangic noktası
|-- models.py       # Veritabanı modelleri
|-- routes.py       # API endpoint'leri
|-- .env            # Çevresel değişkenler (gizli)
|-- requirements.txt # Bağımlılıklar
```

### 5.2 Veritabanı Modeli (Task)

| Alan Adı    | Veri Tipi    | Açıklama                                |
|-------------|--------------|-----------------------------------------|
| id          | Integer (PK) | Benzersiz görev kimliği, otomatik artar |
| title       | String(100)  | Görev başlığı, zorunlu alan             |
| description | String(500)  | Görev açıklaması, opsiyonel             |
| status      | String(20)   | Görev durumu: pending / completed       |
| created_at  | DateTime     | Oluşturma tarihi, otomatik kaydedilir   |

### 5.3 API Endpoint'leri

| Method | Endpoint        | Açıklama              | HTTP Kodu   |
|--------|-----------------|-----------------------|-------------|
| GET    | /api/tasks      | Tüm görevleri listele | 200 OK      |
| POST   | /api/tasks      | Yeni görev oluştur    | 201 Created |
| PUT    | /api/tasks/<id> | Görevi güncelle       | 200 OK      |
| DELETE | /api/tasks/<id> | Görevi sil            | 200 OK      |

### 5.4 Örnek API İsteği ve Yanıtı

POST /api/tasks - Yeni Görev Oluşturma:

```
İstek: {"title": "Flask API geliştir", "description": "Route'lari tamamla"}
Yanıt: {"id": 1, "title": "Flask API geliştir", "description": "Route'lari tamamla", "status": "pending", "created_at": "2026-04-01T12:00:00"}
```

## 6. FRONTEND GELISTIRME (REACT)

### 6.1 React Bileşenleri

| Bileşeni | Açıklama                                                                    |
|----------|-----------------------------------------------------------------------------|
| App      | Ana bileşeni. Görev listesi, ekleme formu ve istatistik kartlarını yönetir. |
| Pomodoro | Bağımsız zamanlayıcı bileşeni. Kullanıcı tanımlı süre ile geri sayım yapar. |

### 6.2 React Hooks Kullanımı

| Hook        | Kullanım Amacı                                                          |
|-------------|-------------------------------------------------------------------------|
| useState    | Görev listesi, form alanları ve düzenle modu için state yönetimi        |
| useEffect   | Sayfa yüklendiğinde API'den görevlerin otomatik çekilmesi               |
| useCallback | Pomodoro'da reset fonksiyonunun gereksiz yeniden oluşturulmasını önleme |

### 6.3 Pomodoro Zamanlayıcı

Pomodoro Teknigi, Francesco Cirillo tarafından geliştirilen zaman yönetimi metodudur. Uygulamadaki Pomodoro bileşeni su özellikleri sunmaktadır:

- Kullanıcı tanımlı çalışma ve mola süreleri (dakika cinsinden)
- Dairesel SVG progress bar ile görsel geri sayım
- Başlat / Durdur / Sıfırla kontrolleri
- Otomatik çalışma-mola geçişi
- Tamamlanan Pomodoro sayacı
- Ayarlar paneli ile süre güncelleme

### 6.4 Arayüz Özellikleri

- Dark mode tasarım (koyu tema)
- İstatistik kartları (Toplam, Bekliyor, Tamamlandı)
- Görev ekleme, düzenleme, tamamlama ve silme
- Enter tuşu ile hızlı görev ekleme
- Animasyonlu hover efektleri



## 7. GUVENLIK ONLEMLERI

Projede aşağıdaki güvenlik önlemleri alınmıştır:

| Önlem                  | Açıklama                                                          |
|------------------------|-------------------------------------------------------------------|
| Environment Variables  | Veritabanı şifreleri .env dosyasında saklanır, GitHub'a yüklenmez |
| AWS Security Groups    | Yalnızca gerekli portlar açıktır (22, 80, 5000, 5432)             |
| CORS Yapılandırması    | Flask-CORS ile kontrollü erişim sağlanmıştır                      |
| Production Modu        | Flask debug=False modunda çalıştırılmaktadır                      |
| SSH Key Authentication | EC2'ye erişim RSA key pair ile sağlanmaktadır                     |
| VPC İzolasyonu         | RDS, AWS VPC içinde çalıştırılmaktadır                            |

## 8. UYGULAMA OZELLIKLERI

| Özellik             | Açıklama                                       | Durum |
|---------------------|------------------------------------------------|-------|
| Görev Ekleme        | Başlık ve açıklama ile yeni görev oluşturma    | Aktif |
| Görev Listeleme     | Tüm görevleri kart şeklinde görüntüleme        | Aktif |
| Görev Düzenleme     | Mevcut görev başlık ve açıklamasını güncelleme | Aktif |
| Görev Tamamlama     | Görev durumunu 'completed' olarak işaretle     | Aktif |
| Görev Silme         | Görevi veri tabanından kalıcı olarak sil       | Aktif |
| İstatistik Kartları | Toplam, bekleyen ve tamamlanan görev sayıları  | Aktif |
| Pomodoro Sayacı     | Ayarlanabilir süreli çalışma zamanlayıcısı     | Aktif |
| Dark Mode           | Koyu tema arayüz tasarımı                      | Aktif |

## 9. TEST VE DOGRULAMA

### 9.1 API Testleri

Backend API'nin tüm endpoint'leri curl komutu kullanılarak test edilmiştir:

| Test                | Komut                                       | Beklenen Sonuç    | Durum |
|---------------------|---------------------------------------------|-------------------|-------|
| GET /api/tasks      | curl<br>http://3.120.139.109:5000/api/tasks | JSON dizi dondu   | Geçti |
| POST /api/tasks     | curl -X POST ... -d {...}                   | 201 Created       | Geçti |
| PUT /api/tasks/1    | curl -X PUT ... -d {...}                    | 200 OK            | Geçti |
| DELETE /api/tasks/1 | curl -X DELETE ...                          | 200 OK            | Geçti |
| RDS Bağlantısı      | Python3 app.py                              | Tablo oluşturuldu | Geçti |
| S3 Erişim           | curl S3 URL                                 | HTML dondu        | Geçti |

### 9.2 Entegrasyon Testi

Frontend ve backend entegrasyonu tarayıcı üzerinden test edilmiştir. React uygulaması S3 adresinden açılmış, görev ekleme, düzenleme, tamamlama ve silme işlemleri gerçekleştirilmiş ve tüm değişikliklerin RDS veritabanına yansıdığı doğrulanmıştır.

## 10. SONUC

Bu proje kapsamında, Amazon Web Services bulut platformu üzerinde çift katmanlı modern bir web uygulaması başarıyla geliştirilmiş ve deploy edilmiştir. Proje, dersin tüm gereksinimlerini karşılamaktadır:

- RESTful API olarak tasarlanmış Flask backend
- React ile geliştirilmiş modern frontend arayüzü
- AWS EC2 üzerinde production-grade deployment (Gunicorn + systemd)
- AWS RDS üzerinde PostgreSQL veritabanı yönetimi
- AWS S3 üzerinde static website hosting
- Yüksek erişebilirlik ve güvenlik önlemleri
- Environment variables ile hassas bilgilerin korunması
- GitHub üzerinde versiyon kontrolü
- Pomodoro zamanlayıcı ve dark mode gibi ek özellikler

Bu proje sayesinde bulut bilişimin temel kavramları olan IaaS, PaaS ve SaaS modelleri pratikte deneyimlenmiştir. AWS servislerinin birbiriyle entegrasyonu, güvenlik grubu yapılandırması ve production ortamına deployment süreci uygulamalı olarak öğrenilmiştir.

| Baglanti       | URL                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Canlı Uygulama | <a href="http://taskflow-frontend-seymakula.s3-website.eu-central-1.amazonaws.com">http://taskflow-frontend-seymakula.s3-website.eu-central-1.amazonaws.com</a> |
| GitHub Repo    | <a href="https://github.com/seymakula/taskflow">https://github.com/seymakula/taskflow</a>                                                                       |
| Backend API    | <a href="http://3.120.139.109:5000/api/tasks">http://3.120.139.109:5000/api/tasks</a>                                                                           |